

MODULE IV

SWINGS

SWING

Swing is a set of Java classes used to create modern graphical user interfaces (GUIs). Before Swing, Java used the **AWT (Abstract Window Toolkit)** to create windows, buttons, text fields, etc. But AWT had many limitations. Swing was introduced as an improved, more flexible GUI framework.

Why Was Swing Needed?

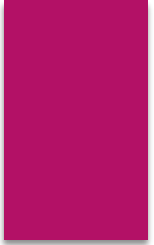
AWT Components Were Heavyweight: AWT components depend on the **native operating system (OS)** to draw UI elements.

Example:

A Java Button → becomes a Windows button on Windows

The same Java Button → becomes a Linux button on Linux

These OS-dependent UI components are called **heavyweight** because they use the OS's windowing system.



Swing was created to solve AWT's problems.

Swing components are:

- **Lightweight**
- **Written entirely in Java**
- **Do not rely on OS peers to draw themselves**

This means:

- Swing looks the same on all platforms
→ "write once, run anywhere" restored.
- Swing components can be customized easily.
- Swing allows complex designs like transparent panels, shaped windows, and advanced graphics.

Swing **does not replace** AWT; instead, it **depends on it**.

1. Swing uses AWT's basic infrastructure

Swing uses **AWT's windowing system** for things like:

- Windows (Frame, Dialog)

- Event handling

- Graphics system

Swing only *extends* these with more powerful components.

2. Swing uses the same event-handling model

Event handling in Swing (mouse, keyboard, window events) is exactly the same as in AWT.

So, you **MUST** understand AWT events to understand Swing.

Two Key Features That Make Swing Better

Swing solves AWT's limitations mainly through:

- 1. Lightweight components**
- 2. Pluggable look and feel**

1. Lightweight components

Every AWT component is tied to a **native OS peer**.

Example:

- AWT Button → Windows Button on Windows
- AWT Button → Linux Button on Linux

Effects:

- Look changes across OS → inconsistent UI
- Cannot do transparency or custom shapes
- More resource usage
- OS decides component behavior → less control

In Java GUI programming, a **lightweight component** is a component that is written **entirely in Java** and **does not depend on the operating system's native GUI elements**.

1. Lightweight components

Except for a few basic containers, **Swing components are completely written in Java.**

This gives many advantages:

1. **No native peers → FULL control in Java**
2. Transparent components are possible
3. Consistent look and behavior across all platforms
4. More efficient & flexible

AWT component = OS draws it ⇒ inconsistent, limited

Swing component = Java draws it ⇒ consistent, customizable

2. Pluggable look and feel

The **Look and Feel (LAF)** determines how GUI components appear on the screen:

Colors

Fonts

Button shapes

Styles of menus, scroll bars, dialogs, etc.

In Swing, the look and feel is **separate from the component's functionality**.

Example: A JButton performs the same actions no matter how it looks.
Only the appearance changes.

COMPONENTS

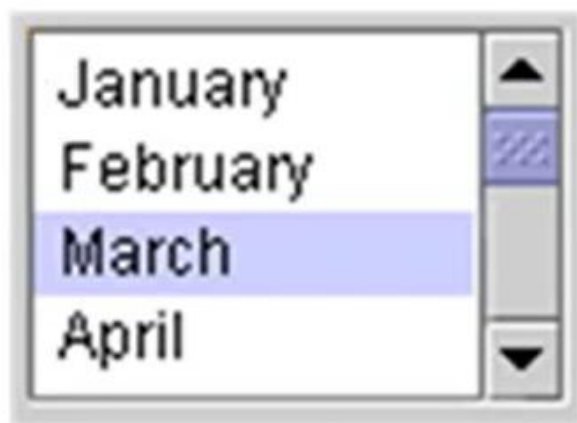
A component is an independent visual control, such as a push button or slider.



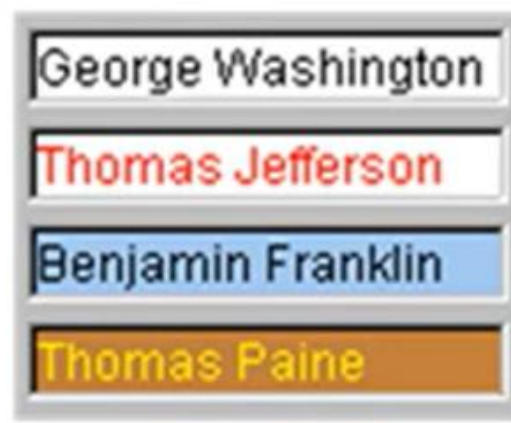
Buttons



Combo Box



List



TextField



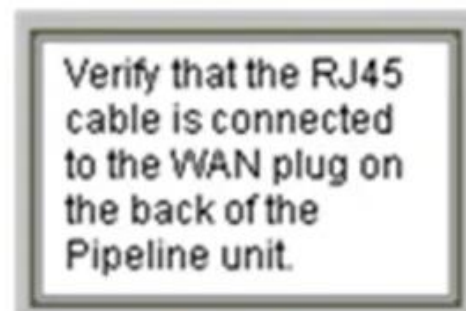
Slider



Menu



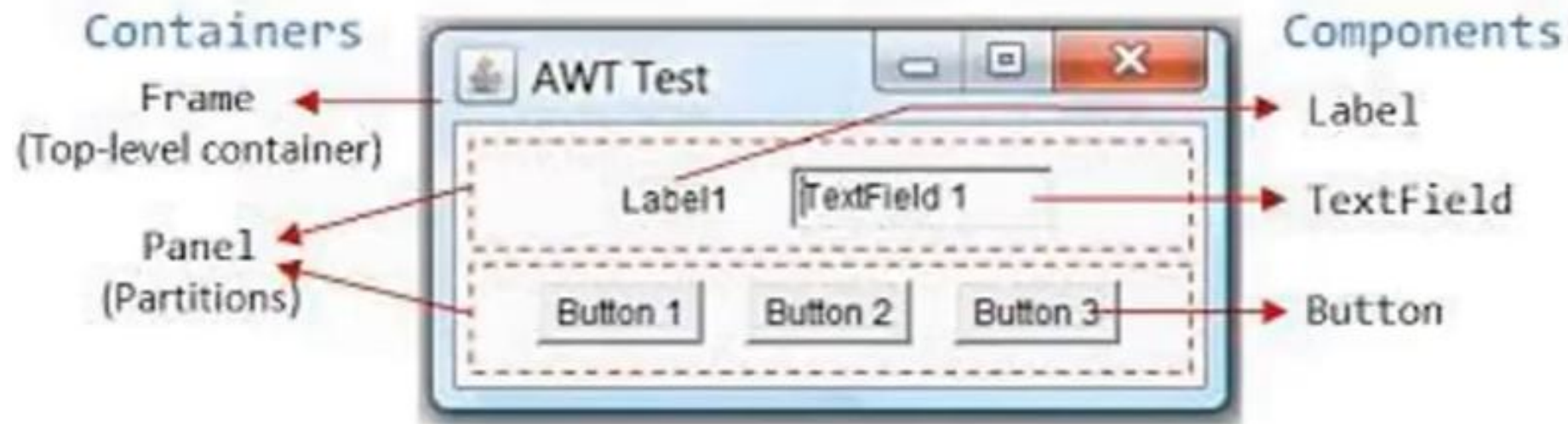
Label



Text Area

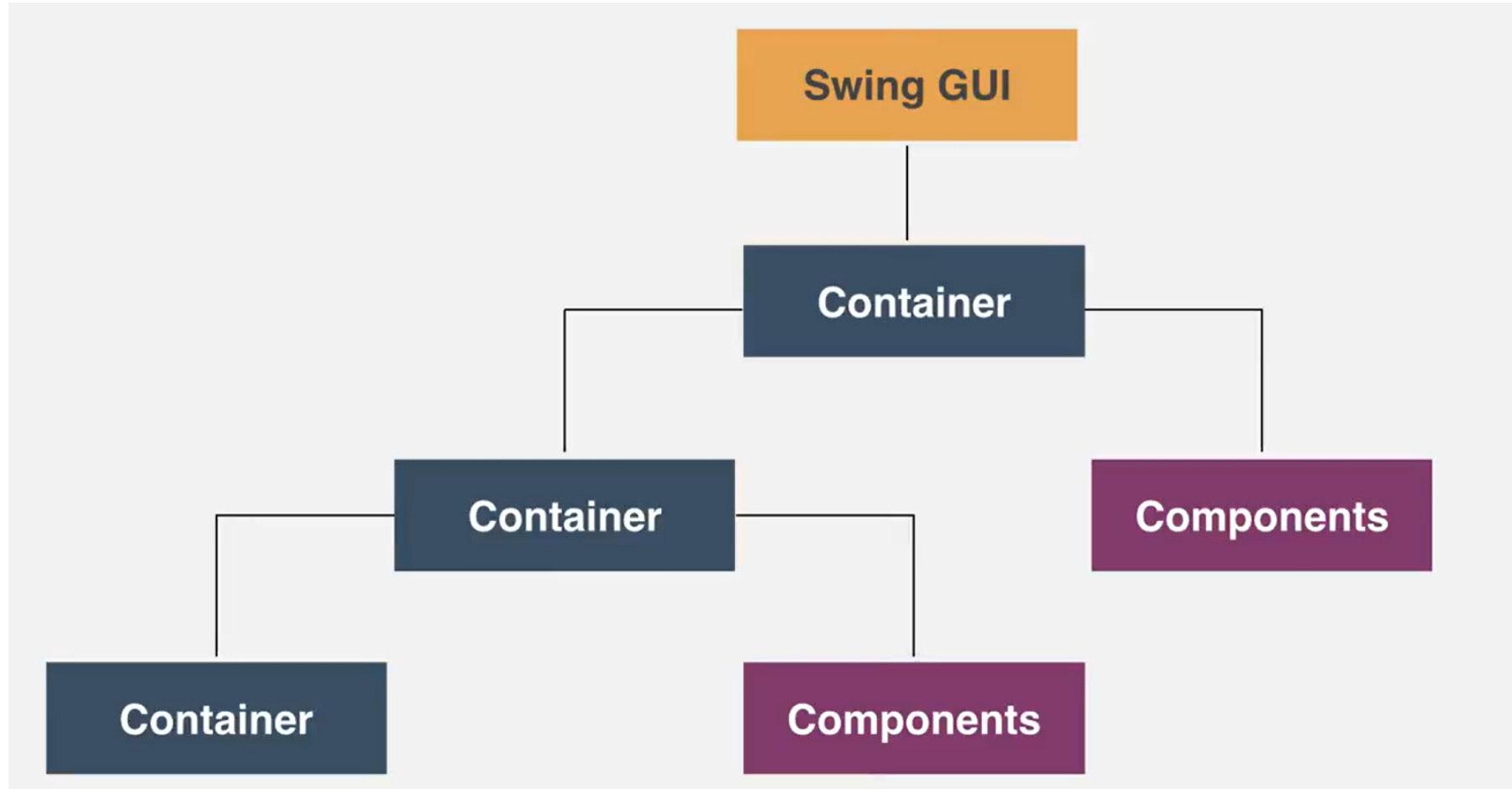
CONTAINERS

A container holds a group of components.



There are 4 top level containers: JFrame, JApplet, JWindow, JDialog

Swing Containment Hierarchy



The Swing Packages

Swing is a very **large subsystem and makes use of many packages**. These are the packages used by Swing that are defined by Java SE 6.

<code>javax.swing</code>	<code>javax.swing.border</code>	<code>javax.swing.colorchooser</code>
<code>javax.swing.event</code>	<code>javax.swing.filechooser</code>	<code>javax.swing.plaf</code>
<code>javax.swing.plaf.basic</code>	<code>javax.swing.plaf.metal</code>	<code>javax.swing.plaf.multi</code>
<code>javax.swing.plaf.synth</code>	<code>javax.swing.table</code>	<code>javax.swing.text</code>
<code>javax.swing.text.html</code>	<code>javax.swing.text.html.parser</code>	<code>javax.swing.text.rtf</code>
<code>javax.swing.tree</code>	<code>javax.swing.undo</code>	

The main package is **javax.swing**. This package must be imported into any program that uses Swing. It contains the classes that implement the basic Swing components, such as push buttons, labels, and check boxes.

A simple Swing application.

```
import javax.swing.*;
```

```
class SwingDemo {
```

```
    SwingDemo() {
```

```
        // Create a new JFrame container.
```

```
        JFrame jfrm = new JFrame("A Simple Swing Application");
```

```
        // Give the frame an initial size.
```

```
        jfrm.setSize(275, 100);
```

```
        // Terminate the program when the user closes the application.
```

```
        jfrm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        // Create a text-based label.
```

```
        JLabel jlab = new JLabel(" Swing means powerful GUIs.");
```

This creates a **top-level window** with:

- ✓ title bar
- ✓ minimize/maximize/close buttons
- ✓ border
- ✓ system menu

This is your application's main window.

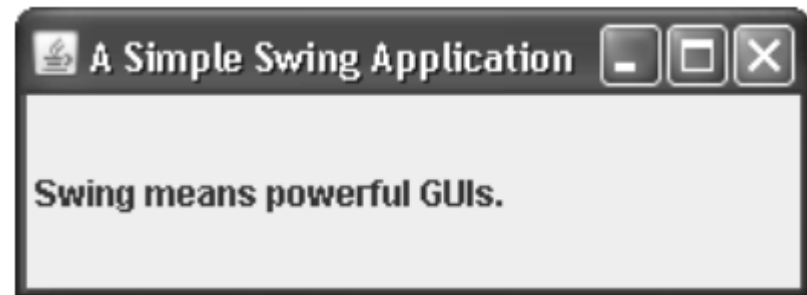
Sets width and height in pixels.

Default action of closing a JFrame = only the window disappears, but program keeps running. We change it so closing the window **terminates the entire program**.

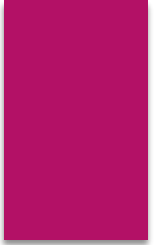
```
// Add the label to the content pane.
jfrm.add(jlab);

// Display the frame.
jfrm.setVisible(true);
}

public static void main(String args[]) {
    // Create the frame on the event dispatching thread.
    SwingUtilities.invokeLater(new Runnable() {
        public void run() {
            new SwingDemo();
        }
    });
}
```



The window produced by the SwingDemo program



```
JLabel jlab = new JLabel(" Swing means powerful GUIs.");  
jfrm.add(jlab);
```

A JLabel is **passive** → displays text or image, no user input.

Old approach (pre-Java 5)
we had to write:

```
jfrm.getContentPane().add(jlab);
```

Today, `add()`, `remove()`, and `setLayout()` automatically operate on the content pane, so the long form is unnecessary.

Event Dispatch Thread (EDT)

All Swing components must be created and updated **on the EDT**, NOT on the main thread.

Swing is **event-driven** and Swing is **not thread-safe**

All event handlers (button clicks, key presses) are processed on the EDT.

If GUI code runs on the main thread, conflicts or deadlocks may occur.

invokeLater()

`SwingUtilities.invokeLater(Runnable r)`

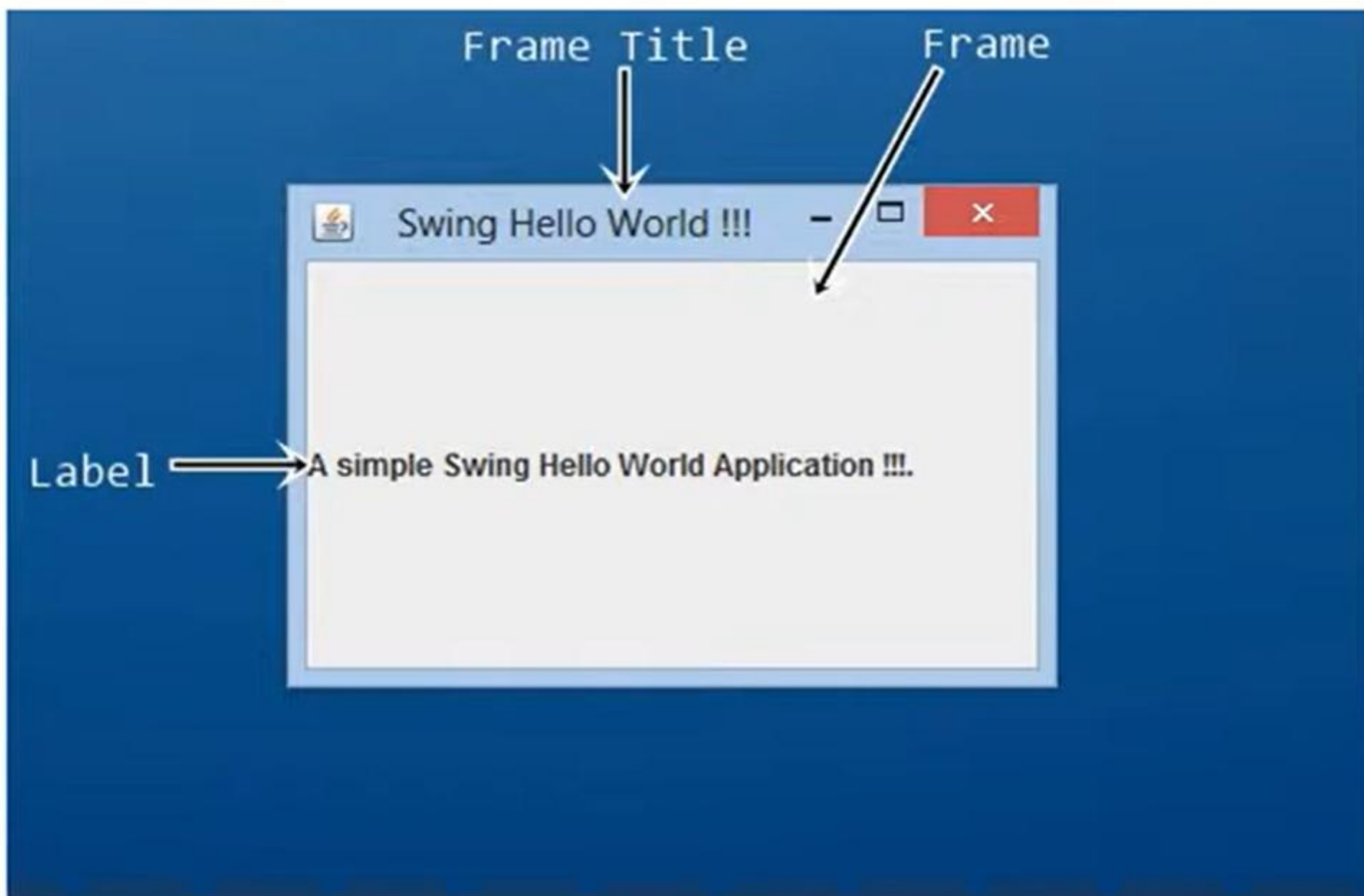
- Schedules GUI creation on the EDT
- Returns immediately
- Most commonly used
- Used in applications

invokeAndWait()

`SwingUtilities.invokeAndWait(Runnable r)`

- Schedules GUI creation on the EDT
- BUT waits for Runnable's run() to finish
- Can throw exceptions
- Required in **applets** (only when initializing GUI)

Both are methods of `SwingUtilities` class



Enumerations

Unlike C++ enums (which are just integer constants), **Java enums are full-fledged classes.**

This means:

- An enum can have **constructors**
- It can have **methods**
- It can contain **instance variables**

An **enumeration (enum)** in Java is a special data type that lets you create a set of **named constants**.

It is created using the **enum** keyword.

```
enum Apple {  
    Jonathan, GoldenDel, RedDel, Winesap, Cortland  
}
```

Each constants is **public**, **static**, and **final** by default.

enum constants which
belongs to the enum Apple

You can create a variable of type Apple like this:

Apple ap;

You don't create enum objects using **new** keyword.
Instead, you simply assign one of the enumeration constants:

ap = Apple.RedDel;

Since ap is of type Apple, it can only store one of the enum's constants.

Comparing Enum Values: Example: **if (ap == Apple.GoldenDel)**

Enums can be used in a switch just like integers or characters.

Example:

```
switch(ap) {  
    case Jonathan:  
        ...  
    case Winesap:  
        ...  
}
```

Printing Enum Constants:

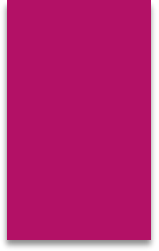
```
System.out.println(Apple.Winesap);
```

Output: Winesap

Example for Enum

```
enum Apple {  
    Jonathan, GoldenDel, RedDel, Winesap, Cortland  
}
```

```
class EnumDemo {  
    public static void main(String args[]) {  
  
        Apple ap;  
  
        ap = Apple.RedDel;  
        System.out.println("Value of ap: " + ap);  
    }  
}
```



```
ap = Apple.GoldenDel;

if(ap == Apple.GoldenDel)
    System.out.println("ap contains GoldenDel.\n");
switch(ap) {
    case Jonathan:
        System.out.println("Jonathan is red.");
        break;
    case GoldenDel:
        System.out.println("Golden Delicious is yellow.");
        break;
    case RedDel:
        System.out.println("Red Delicious is red.");
        break;
    case Winesap:
        System.out.println("Winesap is red.");
        break;
    case Cortland:
        System.out.println("Cortland is red.");
        break;
}
}
```

Output:

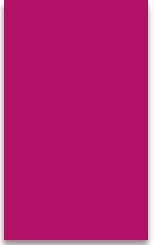
Value of ap: RedDel
ap contains GoldenDel.
Golden Delicious is yellow.

The values() and valueOf() Methods

All enumerations automatically contain two predefined methods: values() and valueOf().

Their general forms are shown here:

```
public static enum-type[ ] values( )  
public static enum-type valueOf(String str)
```



```
Class StringDemo
{
    Public static void main(String[] args)
    {
        float x =123.45f;
        String s1=x;
        System.out.println(s1);
    }
}
```

Output: Error: Incompatible type

```
Class StringDemo
{
    Public static void main(String[] args)
    {
        float x =123.45f;
        String s1=String.valueOf(x);
        System.out.println(s1);
    }
}
```

Output: 123.45

The following program demonstrates the values() and valueOf() methods:

```
// Use the built-in enumeration methods.
// An enumeration of apple varieties.
enum Apple {
Jonathan, GoldenDel, RedDel, Winesap, Cortland
}
class EnumDemo2
{ public static void main(String args[]) {
Apple ap;
System.out.println("Here are all Apple constants:");

// use values()
Apple allapples[] = Apple.values();
for(Apple a : allapples)
System.out.println(a);
System.out.println();

// use valueOf()
ap = Apple.valueOf("Winesap");
System.out.println("ap contains " + ap);
}
}
```

Output

Here are all Apple constants:
Jonathan GoldenDel RedDel Winesap Cortland
ap contains Winesap

Type Wrappers

Java has **primitive data types** like `int`, `double`, `char`, `boolean`, etc.

These are **not objects** and are used for efficiency. But sometimes, you **need an object**, not a primitive.

For this purpose, Java gives **Type Wrapper classes**—classes that “wrap” a primitive inside an object.



List of Type Wrapper Classes

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Character Wrapper Class

```
Character ch = new Character('A');  
char c = ch.charValue(); // unboxing
```

Wraps a char inside an object. `charValue()` extracts the primitive value.

Numeric Wrapper Classes

These wrap numeric primitives:

Byte
Short
Integer
Long
Float
Double

They all extend the **Number** class. Number provides methods like:

```
byteValue()  
shortValue()  
intValue()  
longValue()  
floatValue()  
doubleValue()
```

Example:

```
Integer i = new Integer(42);  
int n = i.intValue();    // unboxing
```

Boxing and Unboxing

Boxing: Converting a primitive → object

Example: Integer iOb = new Integer(100);

Unboxing: Converting an object → primitive

Example: int i = iOb.intValue();

Example:

```
class Wrap {  
    public static void main(String args[]) {  
        Integer iOb = new Integer(100); // boxing  
        int i = iOb.intValue();         // unboxing  
  
        System.out.println(i + " " + iOb);  
        // Output: 100 100  
    }  
}
```

String Handling

1. String objects are immutable: Once a String object is created, its content **cannot be changed**.

example: `String s = "Hello";`

If you try to modify it using: `s.concat(" World");`

It does **not** change the original string. Instead, it creates a **new String object** containing "Hello World".

2. String references are mutable: Even though the **content** of a String object cannot change, the **reference variable** that points to it *can* change.

Example:

```
String s = "Hello"; // s refers to "Hello"
```

```
s = "World";       // now s refers to "World"
```

String Constructors

1. Default String Constructor

```
String s = new String();
```

This creates an **empty string** (""), meaning it contains no characters.

2. Constructing a String from a char[] array

```
String(chars[]) //Constructor
```

```
char chars[] = { 'a', 'b', 'c' };
```

```
String s = new String(chars); // This converts the character array into a String. Result → "abc"
```

3. Constructing a String from a subrange of char[]

String(char chars[], int startIndex, int numChars) //Constructor

startIndex → from where to begin

numChars → how many characters to pick

Example:

```
char chars[] = { 'a', 'b', 'c', 'd', 'e', 'f' };
```

```
String s = new String(chars, 2, 3); //Starting at index 2 (c) and taking 3 characters → cde.
```

4. Constructing a String from another String

```
String(String strObj) //Constructor
```

Example:

```
String s1 = new String("Java");  
String s2 = new String(s1);
```

Both strings contain "Java".

A new String object is created even though the content is same..

5. Constructing Strings from byte arrays

Many old systems and internet protocols use **8-bit ASCII**, not 16-bit Unicode. So Java allows you to create a String from **byte[]** where: Each byte represents an ASCII code (0–127).

Constructors:

```
String(byte ascii[])
```

```
String(byte ascii[], int startIndex, int numChars)
```

Example

```
byte ascii[] = {65, 66, 67, 68, 69, 70};
```

```
String s1 = new String(ascii);           // Output: ABCDEF
```

```
String s2 = new String(ascii, 2, 3);    // Output: CDE
```

6. Extended constructors with specific encodings

Sometimes, a byte does not always represent ASCII.

For example:

UTF-8

ISO-8859-1

UTF-16

etc.

Constructors:

`String(byte bytes[], String charsetName)`

`String(byte bytes[], int start, int length, String charsetName)`

String Length

String Length The length of a string is the number of characters that it contains. To obtain this value, call the `length()` method, shown here:

```
int length( )
```

The following fragment prints “3”, since there are three characters in the string `s`:

```
char chars[] = { 'a', 'b', 'c' };  
String s = new String(chars);  
System.out.println(s.length());
```

Special String Operations

Java gives **extra built-in support** for Strings because they are used so often.

1. **String Literals**
2. **String Concatenation Using +**
3. **Concatenating Strings with Other Data Types**
4. **Be Careful with + and Operator Precedence**
5. **String Conversion Using `valueOf()` and `toString()`**
6. **The `toString()` Method**

1. String Literals:

Normally, you can create a string like this:

```
char chars[] = { 'a', 'b', 'c' };  
String s1 = new String(chars);
```

But Java allows a **much simpler way**:

`String s2 = "abc";` // string literal. Whenever you type text inside double quotes (" "), Java automatically creates a String object for you.

So "abc" itself **is a String object**.

You can even call methods directly on a string literal:

```
System.out.println("abc".length()); // prints 3
```

2. String Concatenation Using +

Java lets you join (concatenate) strings using the **+** operator:

```
String age = "9";  
String s = "He is " + age + " years old.";
```

Output:

He is 9 years old.

```
String longStr = "This could have been " +  
                 "a very long line " +  
                 "that wrapped around.";
```

3. Concatenating Strings with Other Data Types

You can also combine strings with numbers:

```
int age = 9;  
String s = "He is " + age + " years old.";
```

Java automatically converts age (an int) to a string.

4. Be Careful with + and Operator Precedence

```
String s = "four: " + 2 + 2;
```

Output: four: 22

```
String s = "four: " + (2 + 2);    //To do addition first, use parentheses:
```

Output: four: 4

5. String Conversion Using `valueOf()` and `toString()`

When Java converts values to strings (like `int` $\rightarrow$ `String`), it uses the `valueOf()` method internally.

Example:

```
String s = "Value is " + 10;
```

Java does:

```
valueOf(10)  $\rightarrow$  "10"
```

Then concatenates.

6. The toString() Method

Every class in Java has a **toString() method**, inherited from Object.

Example:

```
class Box {  
    double width;  
    double height;  
    double depth;  
  
    Box(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
  
    public String toString() {  
        return "Dimensions are " + width + " by " + depth + " by " + height + ".";  
    }  
}  
  
// this wont work
```

6. The toString() Method

Every class in Java has a **toString() method**, inherited from Object.

Example:

```
Box b = new Box(10, 12, 14);
```

```
System.out.println(b);
```

Because whenever a **non-string** is used in:

- `println()`
- concatenation (+)

Java automatically calls **toString()** on that object.

Character Extraction

1. **charAt(int index):** Returns the character at a specific index.

Example: `char ch = "abc".charAt(1);`
`System.out.println(ch);` `// output: b`

2. **getChars(int start, int end, char[] target, int targetStart)**

Example: `class GetCharsDemo {`
 `public static void main(String[] args) {`
 `String s = "This is a demo of getChars method.";`
 `int start = 10;`
 `int end = 14;`

`char[] buf = new char[end - start];`
 `s.getChars(start, end, buf, 0);`

`System.out.println(buf); // demo`

`}`
`}`

3. `getBytes()`

Converts the string into a **byte array** using default encoding (ASCII/UTF-8).
Useful when sending data over a network or writing to ASCII files.

Example:

```
byte[] b = "ABC".getBytes();
```

4. `toCharArray()`

Converts **entire string** into a char array.

Example:

```
char[] arr = "Hello".toCharArray();
```

String Comparison

1. `equals( )` and `equalsIgnoreCase( )`

`equals()` — Case-Sensitive Comparison

Returns **true** only if both strings contain exactly the same characters.

Example:

```
String a = "Hello";
```

```
String b = "Hello";
```

```
String c = "HELLO";
```

```
System.out.println(a.equals(b)); // true
```

```
System.out.println(a.equals(c)); // false
```

1. equals() and equalsIgnoreCase()

equalsIgnoreCase() — Case-Insensitive Comparison

```
System.out.println("Hello".equalsIgnoreCase("HELLO")); // true
```

```
// Demonstrate equals() and equalsIgnoreCase().
```

```
class equalsDemo {  
    public static void main(String args[]) {  
        String s1 = "Hello";  
        String s2 = "Hello";  
        String s3 = "Good-bye";  
        String s4 = "HELLO";  
        System.out.println(s1 + " equals " + s2 + " -> " + s1.equals(s2));  
        System.out.println(s1 + " equals " + s3 + " -> " + s1.equals(s3));  
        System.out.println(s1 + " equals " + s4 + " -> " + s1.equals(s4));  
        System.out.println(s1 + " equalsIgnoreCase " + s4 + " -> " + s1.equalsIgnoreCase(s4));  
    }  
}
```

The output from the program is shown here:

Hello equals Hello -> true

Hello equals Good-bye -> false

Hello equals HELLO -> false

Hello equalsIgnoreCase HELLO -> true

2. regionMatches() — The `regionMatches( )` method compares a specific region inside a string with another specific region in another string

```
boolean result = "WelcomeHome".regionMatches(7, "HomeTown", 0, 4);  
System.out.println(result); // true (Home vs Home)
```

3. startsWith() and endsWith()

```
"Football".startsWith("Foot"); // true
```

```
"Football".endsWith("ball"); // true
```

```
"Football".startsWith("ball", 4); // true
```

5. equals() vs ==

equals() - Compares **content** of strings.

== Compares **memory addresses** (references), NOT content.

Example:

```
String s1 = "Hello";
```

```
String s2 = new String("Hello");
```

```
System.out.println(s1.equals(s2)); // true (content same)
```

```
System.out.println(s1 == s2);    // false (different objects)
```

5. compareTo()

```
int compareTo(String str)
```

String comparison happens **character-by-character** using Unicode/ASCII values.

Example

ASCII values:

'A' = 65

'Z' = 90

'a' = 97

'z' = 122

So:

'A' < 'a' (because 65 < 97)

Therefore: "Apple" comes **before** "apple"

This is why **uppercase letters come before lowercase letters**.

5. compareTo()

Example: Bubble Sort Using compareTo()

```
class SortString {  
    static String arr[] = {  
        "Now", "is", "the", "time", "for", "all", "good", "men",  
        "to", "come", "to", "the", "aid", "of", "their", "country"  
    };  
  
    public static void main(String args[]) {  
        for (int j = 0; j < arr.length; j++) {  
            for (int i = j + 1; i < arr.length; i++) {  
                if (arr[i].compareTo(arr[j]) < 0) {  
                    String t = arr[j];  
                    arr[j] = arr[i];  
                    arr[i] = t;  
                }  
            }  
            System.out.println(arr[j]);  
        }  
    }  
}
```

Output begins with:

```
Now  
aid  
all  
come  
country  
...
```

"Now" starts with 'N'
'N' has ASCII value **78**
All other words start with
lowercase letters ('a' = 97, 'c'
= 99, 'm' = 109, etc.)
Since **78 < 97**, "Now" is
considered **smaller** and
appears first.

5. compareToIgnoreCase(): If you don't want uppercase/lowercase to affect sorting, use: If you replace compareTo() with compareToIgnoreCase() in the bubble sort:

Example: Bubble Sort Using compareTo()

```
class SortString {
    static String arr[] = {
        "Now", "is", "the", "time", "for", "all", "good", "men",
        "to", "come", "to", "the", "aid", "of", "their", "country"
    };

    public static void main(String args[]) {
        for (int j = 0; j < arr.length; j++) {
            for (int i = j + 1; i < arr.length; i++) {
                if (arr[i].compareToIgnoreCase(arr[j]) < 0) {
                    String t = arr[j];
                    arr[j] = arr[i];
                    arr[i] = t;
                }
            }
            System.out.println(arr[j]);
        }
    }
}
```

Output begins with:

```
aid
all
come
country
for
good
...
Now
```

...
"Now" will **not** come first because 'N' and 'n' are treated the same.

Searching Strings

The String class provides two methods that allow you to search a string for a specified character or substring:

- **indexOf()** Searches for the first occurrence of a character or substring.
- **lastIndexOf()** Searches for the last occurrence of a character or substring.

Example string: **"Now is the time for all good men to come to the aid of their country."**

`indexOf('t') = 7`

`lastIndexOf('t') = 65`

`indexOf("the") = 7`

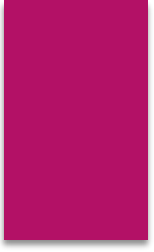
`lastIndexOf("the") = 55`

`indexOf('t', 10) = 11` //Start searching for 't' **from index 10 onward**, The first 't' after index 10 is at **11**.

`lastIndexOf('t', 60) = 55` //search **backwards** from index 60.

`indexOf("the", 10) = 44`

`lastIndexOf("the", 60) = 55`



```
// Demonstrate indexOf() and lastIndexOf().
class indexOfDemo {
public static void main(String args[]) {
    String s = "Now is the time for all good men " + "to come to the aid of their country.";
    System.out.println(s);
    System.out.println("indexOf(t) = " + s.indexOf('t'));
    System.out.println("lastIndexOf(t) = " + s.lastIndexOf('t'));
    System.out.println("indexOf(the) = " + s.indexOf("the"));
    System.out.println("lastIndexOf(the) = " + s.lastIndexOf("the"));
    System.out.println("indexOf(t, 10) = " + s.indexOf('t', 10));
    System.out.println("lastIndexOf(t, 60) = " + s.lastIndexOf('t', 60));
    System.out.println("indexOf(the, 10) = " + s.indexOf("the", 10));
    System.out.println("lastIndexOf(the, 60) = " + s.lastIndexOf("the", 60));
} }
```

Here is the **output** of this program:

Now is the time for all good men to come to the aid of their country.

indexOf(t) = 7

lastIndexOf(t) = 65

indexOf(the) = 7

lastIndexOf(the) = 55

indexOf(t, 10) = 11

lastIndexOf(t, 60) = 55

indexOf(the, 10) = 44

lastIndexOf(the, 60) = 55

Modifying a String

In Java, **String objects cannot be changed** after they are created. This is called **immutability**.

So, when you want to *modify* a string, you have two choices:

1. Use StringBuffer or StringBuilder
2. Use String methods that create a new string

substring()

The following program uses **substring()** to replace all instances of one substring with another within a string:

```
// Substring replacement.
class StringReplace {
public static void main(String args[]) {
String org = "This is a test. This is, too.";
String search = "is";
String sub = "was";
String result = "";
int i;
do {
// replace all matching substrings
System.out.println(org);
i = org.indexOf(search);
if(i != -1) {
result = org.substring(0, i);
```

substring()

```
result = result + sub;  
result = result + org.substring(i + search.length());  
org = result; } } while(i != -1);  
} }
```

The **output** from this program is shown here:

This is a test. This is, too.

Thwas is a test. This is, too.

Thwas was a test. This is, too.

Thwas was a test. Thwas is, too.

Thwas was a test. Thwas was, too.

concat()

The concat() method is used to **append one string to another**. It creates a **new String object**, because strings in Java are **immutable**.

Syntax: String concat(String str)

Example 1: String s1 = "one";
String s2 = s1.concat("two");

How it works:

- The string "one" is in s1.
- Calling s1.concat("two") creates a new string by joining "two" at the end of "one".
- So "one" + "two" becomes "onetwo".
- This new value is stored in s2.
- concat() does **not** change s1, because strings are immutable.
s1 is still "one"; s2 becomes "onetwo".

Example 2: String s1 = "one";
String s2 = s1 + "two"; //this works same as example 1

replace()

This version of replace() is used to replace **a single character** with **another character** throughout the entire string.

Syntax: String replace(char original, char replacement)

Example: String s = "Hello".replace('l', 'w');

How it works:

"**Hello**" contains two '**l**' characters.

Both are replaced with '**w**'.

The returned value is "**Hewwo**".

Example program

```
import java.io.*;

class TrimExample {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter your name (with extra spaces): ");
        String name = br.readLine();

        // Remove leading and trailing spaces
        String trimmedName = name.trim();

        System.out.println("Original input: [" + name + "]");
        System.out.println("After trim():  [" + trimmedName + "]");
    }
}
```

Output:

```
Enter your name (with extra spaces):  John Doe
Original input: [  John Doe  ]
After trim():  [John Doe]
```

Data Conversion Using `valueOf()`

`valueOf()` - `String.valueOf()` converts **any data type** (primitive or object) into its **string (text) representation**.

It is a **static, overloaded** method in the `String` class.

Because it is overloaded, Java provides different versions to handle different data types.

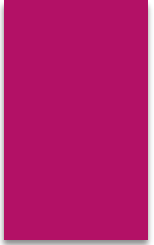
Common Overloaded Forms:

`static String valueOf(double num)`

`static String valueOf(long num)`

`static String valueOf(Object ob)`

`static String valueOf(char chars[])`



For primitive types (int, long, float, double, boolean):

Java returns their standard text form.

Example:

`String.valueOf(10.5)` → "10.5"

For objects:

Java calls the object's `toString()` method internally.

Example:

`String.valueOf(myObject)` → `myObject.toString()`

For char arrays (`char[]`):

Java creates a new `String` containing all characters in the array.

Example:

`char[] c = {'H','i'};`

`String.valueOf(c)` → "Hi"

Changing the Case of Characters Within a String

Java provides two built-in `String` methods to convert the case of characters:

`toLowerCase()`:

Converts **all alphabetic characters** in the string to **lowercase**.
Non-alphabetic characters (digits, punctuation, spaces) remain unchanged.

`toUpperCase()`

Converts **all alphabetic characters** in the string to **uppercase**.
Non-alphabetic characters remain unchanged.

Both methods return a **new String**, because Java strings are immutable.

Example:

```
class ChangeCase {  
    public static void main(String args[]) {  
        String s = "This is a test.";  
  
        System.out.println("Original: " + s);  
  
        String upper = s.toUpperCase();  
        String lower = s.toLowerCase();  
  
        System.out.println("Uppercase: " + upper);  
        System.out.println("Lowercase: " + lower);  
    }  
}
```

Output:

Original: This is a test.
Uppercase: THIS IS A TEST.
Lowercase: this is a test.

Additional String Methods

Method	Description
<code>int codePointAt(int <i>i</i>)</code>	Returns the Unicode code point at the location specified by <i>i</i> . Added by J2SE 5.
<code>int codePointBefore(int <i>i</i>)</code>	Returns the Unicode code point at the location that precedes that specified by <i>i</i> . Added by J2SE 5.
<code>int codePointCount(int <i>start</i>, int <i>end</i>)</code>	Returns the number of code points in the portion of the invoking String that are between <i>start</i> and <i>end</i> -1. Added by J2SE 5.
<code>boolean contains(CharSequence <i>str</i>)</code>	Returns true if the invoking object contains the string specified by <i>str</i> . Returns false , otherwise. Added by J2SE 5.
<code>boolean contentEquals(CharSequence <i>str</i>)</code>	Returns true if the invoking string contains the same string as <i>str</i> . Otherwise, returns false . Added by J2SE 5.
<code>boolean contentEquals(StringBuffer <i>str</i>)</code>	Returns true if the invoking string contains the same string as <i>str</i> . Otherwise, returns false .

Additional String Methods

<code>static String format(String <i>fmtstr</i>, Object ... <i>args</i>)</code>	Returns a string formatted as specified by <i>fmtstr</i> . (See Chapter 18 for details on formatting.) Added by J2SE 5.
<code>static String format(Locale <i>loc</i>, String <i>fmtstr</i>, Object ... <i>args</i>)</code>	Returns a string formatted as specified by <i>fmtstr</i> . Formatting is governed by the locale specified by <i>loc</i> . (See Chapter 18 for details on formatting.) Added by J2SE 5.
<code>boolean matches(string <i>regExp</i>)</code>	Returns true if the invoking string matches the regular expression passed in <i>regExp</i> . Otherwise, returns false .
<code>int offsetByCodePoints(int <i>start</i>, int <i>num</i>)</code>	Returns the index with the invoking string that is <i>num</i> code points beyond the starting index specified by <i>start</i> . Added by J2SE 5.
<code>String replaceFirst(String <i>regExp</i>, String <i>newStr</i>)</code>	Returns a string in which the first substring that matches the regular expression specified by <i>regExp</i> is replaced by <i>newStr</i> .
<code>String replaceAll(String <i>regExp</i>, String <i>newStr</i>)</code>	Returns a string in which all substrings that match the regular expression specified by <i>regExp</i> are replaced by <i>newStr</i> .

Additional String Methods

Method	Description
<code>String[] split(String <i>regExp</i>)</code>	Decomposes the invoking string into parts and returns an array that contains the result. Each part is delimited by the regular expression passed in <i>regExp</i> .
<code>String[] split(String <i>regExp</i>, int <i>max</i>)</code>	Decomposes the invoking string into parts and returns an array that contains the result. Each part is delimited by the regular expression passed in <i>regExp</i> . The number of pieces is specified by <i>max</i> . If <i>max</i> is negative, then the invoking string is fully decomposed. Otherwise, if <i>max</i> contains a nonzero value, the last entry in the returned array contains the remainder of the invoking string. If <i>max</i> is zero, the invoking string is fully decomposed.
<code>CharSequence subSequence(int <i>startIndex</i>, int <i>stopIndex</i>)</code>	Returns a substring of the invoking string, beginning at <i>startIndex</i> and stopping at <i>stopIndex</i> . This method is required by the CharSequence interface, which is now implemented by String .

StringBuffer

`StringBuffer` is a Java class used to create **modifiable**, **growable** sequences of characters.

It is considered a **peer class** of `String` because both manage character data, but they differ in how they handle modifications.

StringBuffer Constructors

`StringBuffer` defines these four constructors:

- `StringBuffer( )`
- `StringBuffer(int size)`
- `StringBuffer(String str)`
- `StringBuffer(CharSequence chars)`

Example operations in StringBuffer:

append() → adds text at the end

insert() → adds text at a specific index

delete() → removes characters

replace() → replaces a portion of characters

reverse() → reverses the entire content

append()

The `append()` method is used with the **StringBuffer** class to **add (concatenate) data to the end of an existing character sequence**.

Unlike `String`, which is immutable, `StringBuffer` is **mutable**, meaning its contents can be changed without creating a new object.

Example:

```
class appendDemo {  
    public static void main(String args[]) {  
        String s;  
        int a = 42;  
        StringBuffer sb = new StringBuffer(40);  
  
        s = sb.append("a=").append(a).append("!").toString();  
        System.out.println(s);  
    }  
}
```

Output: a = 42!

insert()

The **insert()** method is used to **insert a sequence of characters at a specified position** within an existing StringBuffer object.

Unlike String, which is immutable, StringBuffer is **mutable**, so the insertion modifies the **same object** rather than creating a new one.

Example:

```
class insertDemo {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer("I Java!");  
        sb.insert(2, "like ");  
        System.out.println(sb);  
    }  
}
```

Output: "I like Java!"

delete() and deleteCharAt()

Example:

```
// Demonstrate delete() and deleteCharAt()  
class deleteDemo {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer("This is a test.");  
        sb.delete(4, 7);  
        System.out.println("After delete: " + sb);  
        sb.deleteCharAt(0);  
        System.out.println("After deleteCharAt: " + sb);  
    }  
}
```

Index: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
Chars: T h i s i s a t e s t .

Output: After delete: This a test.
After deleteCharAt: his a test.

reverse()

// Using reverse() to reverse a StringBuffer.

```
class ReverseDemo {  
    public static void main(String args[]) {  
        StringBuffer s = new StringBuffer("abcdef");  
        System.out.println(s);  
        s.reverse();  
        System.out.println(s);  
    }  
}
```

Here is the output produced by the program:

```
abcdef  
fedcba
```

replace()

```
// Demonstrate replace()  
class replaceDemo {  
    public static void main(String args[]) {  
        StringBuffer sb = new StringBuffer("This is a test.");  
        sb.replace(5, 7, "was");  
        System.out.println("After replace: " + sb);  
    }  
}
```

Here is the output produced by the program:
After replace: **This was a test.**

StringBuilder

The `StringBuilder` class is similar to `StringBuffer` but not synchronized, making it faster in single-threaded applications. It was introduced in Java 5 as an alternative for better performance.

Key Points:

- Mutable (just like `StringBuffer`).
- Not synchronized → Not thread-safe.
- Faster than `StringBuffer`.
- Best for single-threaded use cases.

Example:

```
class StringBuilderExample {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder("Java");  
        sb.append(" Programming");           // sb = "Java Programming"  
        sb.replace(0, 4, "C++");             //sb = "C++ Programming"  
        sb.delete(3, 6);                     // " Pr" removed sb = "C++ogramming"  
        sb.reverse();                         // sb = "gnimmargom++C"  
        System.out.println(sb);              // Output: gnimmargom++C  
    }  
}
```

Comparison: String vs StringBuffer vs StringBuilder



Feature	String (Immutable)	StringBuffer (Mutable)	StringBuilder (Mutable)
Mutability	Cannot change	Can change	Can change
Thread-Safety	(Immutable)	Synchronized (Safe)	Not synchronized
Performance	Medium	Slower (due to sync)	Faster (no sync)
Use Case	Fixed text values	Multi-threaded apps	Single-threaded apps